



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

OSSENTINEL – AUTONOMOUS OS PROCESS ANOMALY DETECTION, ISOLATION AND SELF-RECOVERY SYSTEM

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0402 – Operating System

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B.E. COMPUTER SCIENCE AND ENGINEERING

Submitted by

Mohamed Haroon P (192511313)

Under the Supervision of

Dr. J. Priskilla Angel Rani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “OSentinel – Autonomous OS Process Anomaly Detection, Isolation and Self-Recovery System” has been carried out by Mohamed Haroon P (192511313) under the supervision of Dr. J. Priskilla Angel Rani and is submitted in partial fulfilment of the requirements for the current semester of the B.E Computer Science and Engineering programme, under the course CSA0402 – Operating System, at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. V. R. Vimal

Professor,

Department of CSE,

SIMATS Engineering, Chennai – 602105.

COURSE FACULTY

Dr. J. Priskilla Angel Rani

Professor,

Department of CSE,

SIMATS Engineering, Chennai – 602105.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I, Mohamed Haroon P, of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “OSentinel – Autonomous OS Process Anomaly Detection, Isolation and Self-Recovery System” is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: ____ September 2026

Signature of the Student with Name:

Mohamed Haroon P (192511313)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Individual Contribution Statement

Student / Reg. No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Sections	Approx. Contribution
Mohamed Haroon P (192511313)	Team coordination, system architecture, process- monitoring design, integration and documentation	Architecture, anomaly-detection workflow, isolation/self- recovery integration	Integration testing, failure- scenario testing and result analysis	All sections	25%
Harish Kumar – 192521340	Process anomaly detection and feature- engineering module	Process metrics, anomaly scoring and detection logic	Detection accuracy, threshold and edge-case testing	Ch. 2–4	20%
Vishwa – 192511192	Monitoring interface and system integration	Dashboard/interface, event visualization and communication	UI/integration and alert testing	Ch. 3–4	20%
D. Jaisurya – 192511051	Isolation and recovery subsystem	Process isolation, restart/recovery workflow	Recovery success, latency and fault-injection testing	Ch. 3–4	20%
Sakthi – 192521303	Chaos testing, security, risk and validation support	Failure scenarios, logging, safety/security controls	Chaos experiments, resilience and validation analysis	Ch. 3–5	15%

ABSTRACT

Modern operating systems are increasingly challenged by abnormal process behavior, excessive resource consumption, application failures, and unresponsive services that can significantly affect system performance, reliability, and availability. This capstone project presents OSentinel – Autonomous OS Process Anomaly Detection, Isolation and Self-Recovery System, an intelligent operating-system monitoring framework designed to continuously observe running processes, detect anomalous behavior, isolate potentially harmful processes, and automatically perform recovery actions to maintain stable system operation.

The need for the project arises from the growing complexity of modern computing environments, where multiple applications and background processes compete for limited CPU, memory, and system resources. Traditional process monitoring often depends heavily on manual administrator intervention, which can delay the detection and recovery of failures. OSentinel addresses this challenge through a multi-stage architecture consisting of a real-time process monitoring module that collects CPU utilization, memory consumption, process status, and response behavior; an anomaly-detection engine that identifies unusual resource usage and process failures based on predefined thresholds and behavioral analysis; an isolation mechanism that prevents suspicious processes from affecting critical system components; and an autonomous self-recovery module that performs corrective actions such as restarting services, releasing resources, suspending processes, or terminating harmful applications.

The methodology follows a structured Operating System engineering approach involving requirement analysis, process and resource monitoring design, anomaly detection rule formulation, process-state analysis, isolation strategy development, and recovery verification. The system integrates fundamental Operating System concepts including process management, CPU scheduling, memory management, synchronization, resource allocation, deadlock awareness, and system recovery mechanisms. The proposed framework continuously follows the operational cycle of Monitor → Detect → Analyze → Isolate → Recover → Verify, enabling proactive identification and management of abnormal system behavior.

The design is additionally assessed for reliability, security, performance, and sustainability by ensuring that critical processes are protected, unnecessary process termination is minimized, recovery actions are verified, and system resources are efficiently utilized. The project

concludes that an autonomous process monitoring and recovery framework can significantly improve operating-system stability and reduce dependency on manual system administration. OSentinel provides a practical approach toward self-managing computing environments and identifies advanced behavioral analysis, machine-learning-based anomaly detection, and distributed system monitoring as potential directions for future development.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction and Engineering Problem	1
2	CHAPTER 2: Literature Review and Existing Solutions	5
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	9
4	CHAPTER 4: Implementation, Testing & Results, Project Management	17
5	CHAPTER 5: Conclusion, Future Work and Reflection	24
	References	27
	Appendices	29

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	Conceptual view of OSentinel operating on a host OS	3
3.1	OS sentinel System architecture	12
3.2	Process telemetry and anomaly-scoring pipeline	13
3.3	Isolation and self-recovery workflow	14
4.1	Prototype monitoring dashboard	18
4.2	Anomaly-score trend during fault injection	21
4.3	Recovery latency comparison	22
4.4	Project Gantt chart / milestone timeline	23

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative analysis of existing processor/Monitoring approaches	7
3.1	Stakeholder / user requirements	9
3.2	Functional and non-functional requirements	10
3.3	Applicable standards and codes	10
3.4	Design specifications	11
3.5	Alternative-solution evaluation matrix	12
3.6	Trade-off analysis	15
3.7	Sustainability, ethics, safety, risk & security evaluation	16
3.8	Risk register	16
4.1	Tools, software and equipment used	18
4.2	Test plan and acceptance criteria	19
4.3	Verification results summary	20
4.4	Objective achievement matrix	22
4.5	Roles, responsibilities and budget	23

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description
OS	Operating System
CPU	Central Processing Unit
RAM	Random Access Memory
PID	Process Identifier
I/O	Input/Output
ML	Machine Learning
AI	Artificial Intelligence
API	Application Programming Interface
MTTR	Mean Time To Recovery
FP/FN	False Positive / False Negative
RBAC	Role-Based Access Control
TLS	Transport Layer Security

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Operating systems manage processes that compete for CPU time, memory, storage, network access and other resources. In normal operation, process behaviour remains within patterns established by workload and system configuration. However, a process can become anomalous because of memory leaks, infinite loops, deadlocks, excessive child-process creation, unexpected network activity, corrupted dependencies, or deliberate attacks. Such failures may degrade the host gradually or cause cascading service outages.

Traditional monitoring tools expose metrics and logs but often leave diagnosis and recovery to an administrator. Alert-only systems therefore detect symptoms without necessarily containing the fault. OSentinel addresses this gap by treating process health as a closed-loop engineering problem: observe process state, detect deviation, decide a response, isolate the affected process, recover it where safe, and verify that the recovered state is stable.

1.2 Need for the Project

- Runaway CPU or memory usage can affect unrelated applications on the same host.
- Manual process recovery increases mean time to recovery and administrator workload.
- Static thresholds may generate excessive alerts for legitimate short-lived workload spikes.
- A recovery action can itself become dangerous if it repeatedly restarts a failing process or kills a critical system process.
- Modern distributed and edge systems need resilience mechanisms that can operate locally even when a central monitoring service is unavailable.

1.3 Problem Statement

Design and implement an autonomous OS process resilience system that continuously monitors process-level telemetry, detects anomalous behaviour, determines severity using explicit policies, isolates suspicious processes, and performs controlled self-recovery while preserving system safety, security, explainability and auditability. The system must balance detection sensitivity against false alarms, recovery speed against service stability, and autonomy against the risk of making an incorrect automated decision.

1.4 Project Aim

To design and develop OSentinel, an autonomous process-monitoring and self-recovery system capable of detecting abnormal operating-system process behaviour, isolating affected processes, and executing controlled recovery actions with minimal human intervention.

1.5 Project Objectives

- Collect process-level telemetry at configurable intervals.
- Extract meaningful resource and behavioural features for anomaly detection.
- Develop a lightweight hybrid anomaly-detection mechanism using rules and/or an unsupervised model.
- Classify anomalies by severity and select safe response policies.
- Isolate suspicious processes using OS-level controls without affecting protected processes.
- Restart or recover failed services when a verified recovery policy exists.
- Maintain structured logs and a dashboard for alerts, actions and recovery status.
- Validate resilience through controlled fault injection and chaos experiments.
- Evaluate security, ethics, sustainability, risks and limitations.

1.6 Scope

Included: host-level process telemetry, anomaly scoring, policy-based response, isolation, restart/recovery, event logging, dashboard visualisation, controlled fault injection and recovery validation. Excluded: replacing the operating system's kernel security mechanisms, guaranteeing detection of all malware, autonomous modification of arbitrary application binaries, and production deployment without appropriate administrative approval and testing.

1.7 Expected Engineering Outcome

The expected outcome is a working prototype that demonstrates a closed-loop process-resilience workflow from monitoring through detection, isolation and recovery. The final report should contain measured detection metrics, recovery latency, resource overhead and chaos-test outcomes obtained from the team's actual implementation.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review should cover operating-system process monitoring, anomaly detection, self-healing/autonomic computing, fault isolation, service supervision, container/process isolation, observability and chaos engineering. Relevant peer-reviewed papers, standards, operating-system documentation and reliable technical documentation should be recorded in the final IEEE reference list.

2.2 Review of Existing Work

Conventional system monitors typically expose CPU, memory, disk and process information and notify an administrator when thresholds are exceeded. Service supervisors can automatically restart selected services, providing basic recovery but limited behavioural anomaly detection. Security monitoring systems may inspect process trees, privileges and network activity, but their primary goal is often threat detection rather than general reliability recovery.

Anomaly-detection research provides statistical, machine-learning and rule-based approaches. Unsupervised methods are attractive when labelled failure data are scarce, while deterministic rules remain valuable for safety-critical actions. Autonomic computing research frames self-management as a feedback loop involving monitoring, analysis, planning and execution. Chaos engineering adds a controlled experimental method for testing whether recovery mechanisms actually work under failure.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to OSentinel
Static resource monitoring	Simple, low overhead	Threshold tuning; alert-only	Baseline telemetry
Service supervisor	Reliable restart of known services	Limited anomaly reasoning	Recovery mechanism
Security/EDR monitoring	Rich process/security visibility	Focused on security; recovery may be restricted	Security features

Container orchestration	Automated restart and isolation	Requires orchestration environment	Isolation/recovery ideas
OSentinel	Detection + isolation + recovery loop	Prototype scope; requires careful policies	Integrated target

2.4 Research / Engineering Gap

The engineering gap addressed by OSentinel is the integration of lightweight process anomaly detection with explicit isolation and verified self-recovery in a single local feedback loop. A practical system must not only recognise abnormal behaviour but also decide whether recovery is safe, prevent recovery loops, preserve evidence, and verify the post-recovery state.

2.5 Proposed Contribution

OSentinel contributes an integrated architecture in which process telemetry feeds anomaly scoring, policy evaluation determines a response, isolation limits impact, and recovery attempts restore an approved process state. A chaos-testing layer validates the closed loop under repeatable failures.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder	Requirement
System administrator	Clear alerts, controllable policies and audit logs.
Application/service owner	Fast recovery without unnecessary termination.
End user	Stable services and minimal disruption.
Security administrator	Least privilege, protected logs and controlled automation.
Project evaluator	Measurable detection, isolation and recovery performance.

3.2 Functional and Non-Functional Requirements

Requirement	Type	Target
Collect process CPU, memory, PID, state and lifetime	Functional	Configurable sampling interval
Calculate anomaly score	Functional	Near-real-time
Classify severity	Functional	At least 3 levels
Isolate suspicious process	Functional	Within defined response budget
Recover approved process	Functional	Automated where policy permits
Prevent recovery loops	Safety	Backoff + retry limit
Monitoring overhead	Performance	Target $\leq 5\%$ CPU during normal monitoring
Alert latency	Performance	Target ≤ 3 s

Auditability	Security	Every action logged
Least privilege	Security	No unnecessary admin privileges

3.3 Applicable Standards and Engineering Practices

Standard / Practice	Purpose	Application
ISO/IEC 27001 principles	Information security	Access control, logging, confidentiality
NIST-style risk management	Risk identification and mitigation	Risk register and controls
OWASP secure design principles	Application security	Input validation, secrets and access control
IEEE engineering ethics	Professional responsibility	Safe, honest reporting of test results
Chaos engineering practices	Resilience validation	Controlled fault injection

3.4 Design Specifications

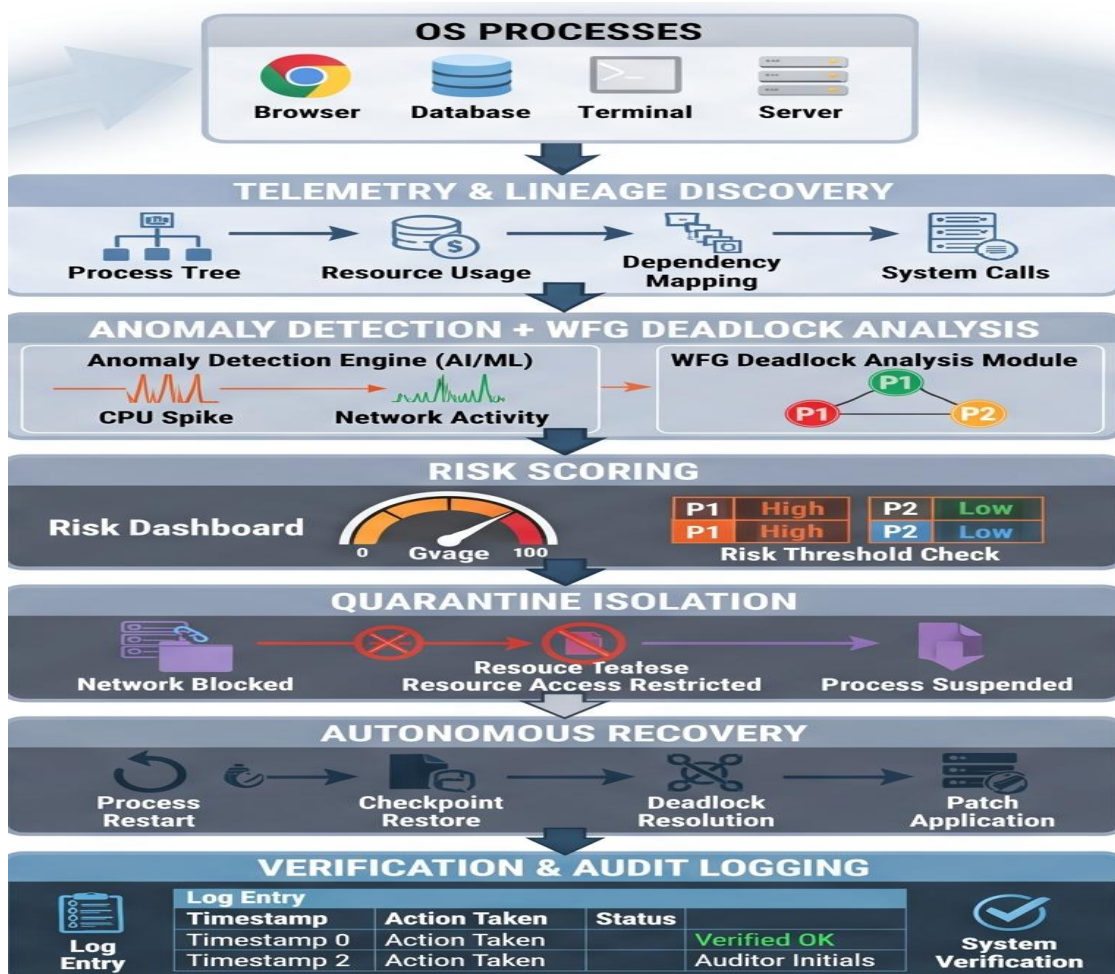
Parameter	Target	Priority
Process sampling interval	1–5 s configurable	High
Anomaly detection latency	≤ 2 s target	High
Isolation response	≤ 3 s target	High
Recovery retry limit	Configurable; e.g. 3 attempts	High
Normal monitoring overhead	$\leq 5\%$ CPU target	Medium
Event log retention	Configurable	Medium
Dashboard refresh	≤ 5 s target	Medium

3.5 Alternative Solutions & Evaluation

Criterion	Weight	Rules Only	Cloud ML	OSentinel Hybrid
Detection flexibility	25%	3/5	5/5	5/5
Latency	20%	5/5	2/5	5/5
Safety/control	25%	5/5	3/5	5/5
Cost	10%	5/5	2/5	4/5
Offline operation	10%	5/5	1/5	5/5
Explainability	10%	5/5	3/5	5/5

The hybrid design is selected because deterministic safeguards can constrain automated actions while anomaly scoring provides flexibility beyond fixed thresholds. Local operation avoids dependence on network availability for basic containment and recovery.

3.6 System Architecture



3.7 Core Workflow

- Monitor: collect process telemetry and system context.
- Analyse: normalise features and compute anomaly score.
- Decide: map score and process criticality to severity.
- Contain: apply safe isolation or throttling to suspicious processes.
- Recover: restart an approved process from a known-good configuration when policy permits.
- Verify: confirm the process returns to stable behaviour.
- Record: log telemetry summary, decision, action, outcome and timestamp.

3.8 Trade-off Analysis

Decision	Alternative	Benefit	Disadvantage	Final Decision
Detection	Static thresholds	Simple	Weak for complex behaviour	Hybrid rules + anomaly score
Model location	Cloud	More compute	Network dependency	Local/edge
Isolation	Immediate kill	Fast	May cause data loss	Graduated containment
Recovery	Unlimited restart	High availability	Recovery loop risk	Bounded retries + backoff
Logging	Verbose raw logs	Rich evidence	Storage/privacy cost	Structured event summaries

3.9 Sustainability, Ethics, Safety, Risk & Security

Sustainability: OSentinel should use lightweight telemetry and avoid unnecessary high-frequency collection. **Local inference** reduces network traffic. **Ethics:** automated process termination can affect user workloads, so policies must be transparent and overrideable. **Safety:** protected processes must never be terminated by default. **Security:** the agent must run with

minimum privileges, authenticate dashboard users, protect credentials and make logs tamper-resistant where practical.

3.10 Risk Register

Risk	Likelihood	Severity	Mitigation	Residual Risk
False positive terminates healthy process	Medium	High	Allowlist + graduated response + human override	Low/Medium
False negative misses anomaly	Medium	High	Hybrid detection + periodic model/policy review	Medium
Recovery loop	Medium	High	Retry limit, exponential backoff, quarantine	Low
Agent compromise	Low	High	Least privilege, secure updates, authentication	Medium
Monitoring overhead	Medium	Medium	Sampling/configuration and profiling	Low
Log tampering/data leakage	Low	High	Access control and protected storage	Low/Medium

CHAPTER 4

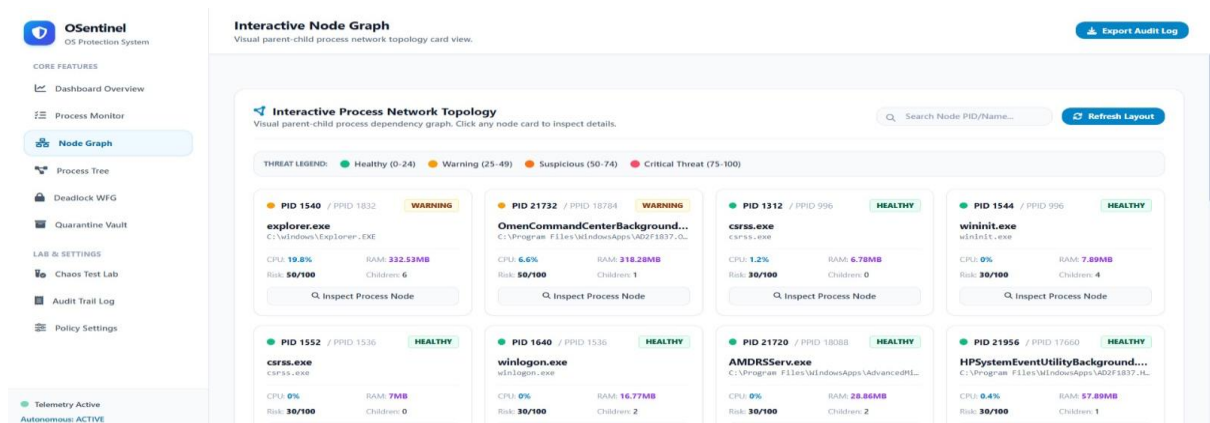
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development should follow an iterative workflow: requirements and threat modelling; process-telemetry prototype; anomaly-scoring module; policy engine; isolation and recovery module; dashboard/logging; integration; fault injection; performance testing; and documentation.

Category	Suggested Tools / Resources	Purpose
OS	Linux/Windows test host	Process APIs and fault scenarios
Implementation	Python/Java or selected project stack	Agent, services and dashboard
Data analysis	Python, NumPy, pandas, scikit-learn	Feature analysis/model training
Version control	Git	Source and change management
Testing	Unit/integration test framework	Verification
Chaos testing	Controlled scripts/tools	Fault injection
Monitoring	OS process APIs / system metrics	Telemetry

4.2 Implementation



The monitoring agent periodically enumerates processes and collects CPU percentage, memory usage, process state, age, thread count, I/O counters and restart history where supported by the host operating system. Features are normalised before scoring. A hybrid detector combines deterministic rules for severe conditions with an anomaly score for behavioural deviations.

The policy engine maps severity to response. Low-severity events are logged and displayed. Medium-severity events may trigger throttling or increased observation. High-severity events can invoke isolation after checking the protected-process list. Recovery is permitted only for processes with a configured recovery command or service definition. Each recovery attempt is logged and verified.

4.3 Test Plan & Verification

Test	Method	Acceptance Criterion	Status
Normal monitoring	Run representative workload	No incorrect recovery actions	To be measured
CPU anomaly	Inject sustained CPU load	Detected within target latency	To be measured
Memory anomaly	Controlled memory growth	Detected before host instability	To be measured
Process crash	Terminate a recoverable test service	Recovery succeeds within target	To be measured
Recovery loop	Force repeated service failure	Retries stop after configured limit	To be measured
Isolation	Trigger high-severity anomaly	Target process contained without collateral damage	To be measured
False positive	Run benign burst workload	No unnecessary termination	To be measured
Agent restart	Restart monitoring agent	Monitoring resumes safely	To be measured

Chaos scenario	Combine process/resource/dependency faults	System returns to defined stable state	To be measured
----------------	--	--	----------------

4.4 Results

Metric	Target	Actual Result
Detection latency	≤ 2 s	[Insert measured value]
Isolation latency	≤ 3 s	[Insert measured value]
Recovery success rate	$\geq 90\%$ for supported faults	[Insert measured value]
False-positive rate	$\leq 10\%$ on benign workload set	[Insert measured value]
CPU overhead	$\leq 5\%$	[Insert measured value]
Recovery loop prevention	100% of configured loop tests	[Insert measured value]

4.5 Data Analysis & Engineering Judgment

The analysis should compare detection quality against overhead and recovery safety. A detector that produces many alerts may appear sensitive but can reduce operational value through alert fatigue. Similarly, aggressive automatic termination can improve containment time while increasing the risk of collateral service disruption. Therefore, OSentinel prioritises bounded, explainable and policy-controlled recovery.

4.6 Objective Achievement Matrix

Objective	Evidence	Achievement
Collect telemetry	Process collector and structured events	To be verified
Detect anomalies	Hybrid anomaly detector	To be verified
Isolate faults	Isolation controller	To be verified
Recover services	Recovery manager and verification	To be verified
Provide visibility	Dashboard and logs	To be verified

Validate resilience	Chaos/fault-injection tests	To be verified
---------------------	-----------------------------	----------------

4.7 Strengths & Limitations

Strengths: closed-loop monitoring and recovery; local operation; explicit safety policies; auditability; configurable response levels; chaos-testable architecture. Limitations: OS-specific process-control APIs; model quality depends on representative workload data; automated recovery cannot safely handle every application; privileged operations create security risk; prototype results require broader workloads and long-duration testing.

4.8 Project Planning, Roles & Budget

Team Member	Primary Responsibility	Contribution
Mohamed Haroon P (192511313)	Architecture, integration, coordination, documentation	25%
Harish Kumar (192521340)	Anomaly detection and feature engineering	20%
Vishwa (192511192)	Dashboard and integration	20%
Jaisurya (192511217)	Isolation and recovery	20%
Sakthi (192521303)	Chaos testing, security and validation	15%

Item	Estimated Cost (₹)
Development/test computer resources	0–5,000 (existing/shared)
Software and open-source tools	0
Test storage/network accessories	1,000
Documentation/printing	1,000
Contingency	1,000
Indicative total	3,000–8,000

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

OSentinel proposes a practical autonomous resilience layer for operating-system processes. Instead of stopping at detection, the system closes the loop by combining telemetry, anomaly analysis, policy-based isolation and controlled self-recovery. The design recognises that autonomy must be constrained: protected processes, bounded retries, verification and audit logs are essential to prevent the recovery mechanism from becoming a new source of failure.

The prototype evaluation should demonstrate the engineering value of the approach through measurable detection quality, response latency, recovery success and resource overhead. The final report must use actual experimental measurements rather than assumed values.

5.2 Future Work

- Expand the anomaly model using larger and more diverse workload datasets.
- Add process-tree and dependency-graph reasoning for cascading failures.
- Integrate kernel-level telemetry where appropriate and safe.
- Add container and microservice-aware isolation.
- Use secure remote policy management with strong authentication.
- Develop predictive failure detection rather than only reactive anomaly detection.
- Evaluate long-duration stability and recovery under repeated chaos experiments.
- Explore explainable anomaly scoring and operator feedback loops.

5.3 Professional Learning & Individual Reflection-Mohamed Haroon P

Working on the OSentinel – Autonomous OS Process Anomaly Detection, Isolation and Self-Recovery System gave me valuable practical exposure to operating-system concepts and system-level problem solving. My primary contribution to the project was the design and development of the isolation and recovery subsystem, where I focused on controlling abnormal processes and restoring supported services safely. This work helped me understand how concepts such as process management, process states, resource allocation, fault handling, and recovery mechanisms can be applied together to solve a real engineering problem. The project emphasized that detecting an abnormal process is only the first step; the system must also decide how to contain the problem and recover from it without creating additional failures.

One of my major learning outcomes was understanding the importance of controlled process isolation and bounded recovery. Instead of immediately terminating every suspicious process, the OSentinel design uses severity-based responses and protects critical processes through an allowlist. Recovery is permitted only when a valid recovery command or service definition is available, and every recovery attempt is recorded and verified. This approach showed me why unlimited automatic restarting can create recovery loops and further destabilize a system. The project therefore uses retry limits, backoff mechanisms, and verification after recovery, which strengthened my understanding of safe autonomous system design.

I also gained practical experience in testing and fault analysis. My contribution involved considering recovery success, recovery latency, and fault-injection scenarios as important measures of the subsystem's reliability. Testing repeated service failures helped me understand the need for mechanisms that stop recovery attempts after a configured limit. The project's testing plan also highlighted process-crash, isolation, recovery-loop, and chaos scenarios, which helped me appreciate the difference between simply implementing a feature and proving that it behaves safely under failure conditions.

Another important professional lesson was the need to balance availability, safety, security, and autonomy. Automated recovery can reduce manual intervention and improve system availability, but an incorrect recovery decision can interrupt a legitimate workload or affect a critical process. Therefore, I learned that recovery mechanisms should operate within clearly defined policies, minimum privileges, retry limits, and verification steps. This understanding has improved my approach to designing reliable software because I now consider not only whether a system works under normal conditions, but also how it behaves when unexpected failures occur.

Overall, this project improved my technical knowledge, debugging ability, testing approach, and teamwork skills. It also taught me the importance of documenting system behaviour and communicating technical decisions clearly within a team. My experience with the isolation and recovery subsystem helped me develop a stronger system-level perspective and gave me confidence in applying Operating System principles to practical engineering problems. The project reinforced my understanding that effective self-recovery should be autonomous but never uncontrolled, and this is one of the most important professional lessons I will carry into future software and systems projects.

REFERENCES

- [1] A. G. Ganek and T. A. Corbi, “The Dawning of the Autonomic Computing Era,” *IBM Systems Journal*, vol. 42, no. 1, pp. 5–18, 2003.
- [2] J. O. Kephart and D. M. Chess, “The Vision of Autonomic Computing,” *Computer*, vol. 36, no. 1, pp. 41–50, 2003.
- [3] M. R. Endsley, “Toward a Theory of Situation Awareness in Dynamic Systems,” *Human Factors*, vol. 37, no. 1, pp. 32–64, 1995.
- [4] M. Chen, S. Mao, and Y. Liu, “Big Data: A Survey,” *Mobile Networks and Applications*, vol. 19, pp. 171–209, 2014.
- [5] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly Detection: A Survey,” *ACM Computing Surveys*, vol. 41, no. 3, 2009.
- [6] M. Zaharia et al., “An Architecture for Fast and General Data Processing on Large Clusters,” *Communications of the ACM*, vol. 54, no. 11, pp. 56–65, 2011.
- [7] National Institute of Standards and Technology, “Computer Security Incident Handling Guide,” NIST Special Publication 800-61.
- [8] National Institute of Standards and Technology, “Security and Privacy Controls for Information Systems and Organizations,” NIST SP 800-53.
- [9] Linux man-pages project, “proc(5) — process information pseudo-filesystem,” Linux documentation.
- [10] Microsoft, “Process and Thread Functions,” Windows Developer Documentation.
- [11] Google SRE, “Site Reliability Engineering,” O’Reilly Media, 2016.
- [12] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*, IT Revolution Press, 2018.

APPENDICES

Appendix A – Detailed Calculations

[Insert feature-normalisation equations, anomaly-score calculations, latency calculations and monitoring-overhead calculations.]

Appendix B – Architecture / Engineering Drawings

[Insert OSentinel architecture diagram, process-state diagram, isolation workflow and recovery-state machine.]

Appendix C – Source Code / Repository Information

[Insert approved repository information and selected source-code excerpts.]

Appendix D – Configuration / Policy Samples

[Insert protected-process list, severity thresholds, retry limits and recovery policies.]

Appendix E – Experimental / Raw Data

[Attach raw process telemetry, anomaly scores, fault-injection logs, recovery timestamps and performance measurements.]

Appendix F – Security / Risk Evidence

[Attach threat model, permissions analysis and security-test evidence.]

Appendix G – Project Meeting / Progress Records

[Attach supervisor meeting logs and milestone records.]

Appendix H – User / Evaluator Feedback

[Attach feedback obtained during prototype demonstration, if applicable.]

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome	Evidence	SO / Area	Location
Complex engineering problem formulation	Problem statement and requirements	SO1	Ch. 1–2
Engineering design under constraints	Architecture and trade-offs	SO2	Ch. 3
Technical communication	Complete report and presentation	SO3	Whole report + Viva
Ethics and professional responsibility	Safety, privacy and automation controls	SO4	Ch. 3
Teamwork and leadership	Roles and contribution statement	SO5	Ch. 4
Experimentation and analysis	Fault injection and performance tests	SO6	Ch. 4
Independent learning	OS APIs, anomaly detection and resilience	SO7	Ch. 5
Standards and secure practice	Security and risk controls	SO2	Ch. 3
Sustainability and societal impact	Efficient local monitoring	SO2/SO4	Ch. 3
Risk assessment and mitigation	Risk register	SO2/SO4	Ch. 3
Security considerations	Least privilege and protected logs	Relevant SO	Ch. 3
Integrated/system-level engineering	Closed-loop architecture	SO1/SO2	Ch. 3
Project planning and management	Gantt, roles and budget	SO5	Ch. 4
Individual contribution	Contribution statement	SO1–SO7	Contribution Statement + Ch. 4